1. Write a program to take a list of 10 farmers, fruits they sell, their cities & the quantity of fruits they have as input. Save the data to notepad before & after sorting based on quantity in tabular manner. Also perform search operation based on farmers' name.

Java Solution: Farmer Data Management

CODE:-

```java
import java.io.FileWriter;

import java.io.IOException;

import java.io.PrintWriter;

import java.util.ArrayList;

import java.util.Collections;

import java.util.Comparator;

import java.util.List;

import java.util.Scanner;

// Class to represent a Farmer

class Farmer {

   String name;

   String fruit;

   String city;

   int quantity;

   public Farmer(String name, String fruit, String city, int quantity) {

      this.name = name;

      this.fruit = fruit;

      this.city = city;

      this.quantity = quantity;

   }
```

```
    // Formats the farmer data into a tabular row
    public String toTableRow() {
        return String.format("%-15s %-15s %-15s %-10d", name, fruit, city, quantity);
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<Farmer> farmers = new ArrayList<>();
        String filename = "farmers_data.txt";
        System.out.println("--- Enter Data for 10 Farmers ---");
        // 1. Take input for 10 farmers
        for (int i = 1; i <= 10; i++) {
            System.out.println("\nFarmer " + i + ":");
            System.out.print("Name: ");
            String name = scanner.nextLine();
            System.out.print("Fruit sold: ");
            String fruit = scanner.nextLine();
            System.out.print("City: ");
            String city = scanner.nextLine();
            System.out.print("Quantity: ");
            int quantity = scanner.nextInt();
            scanner.nextLine(); // Consume the newline character
            farmers.add(new Farmer(name, fruit, city, quantity));
        }
```

```
        // 2. Save data to notepad BEFORE sorting
        saveDataToFile(farmers, filename, "BEFORE SORTING (Based on Input Order)", false);
        // 3. Sort the data based on quantity (Ascending order)
        Collections.sort(farmers, new Comparator<Farmer>() {
            @Override
            public int compare(Farmer f1, Farmer f2) {
                return Integer.compare(f1.quantity, f2.quantity);
            }
        });
        // 4. Save data to notepad AFTER sorting
        saveDataToFile(farmers, filename, "AFTER SORTING (Based on Quantity)", true);
        System.out.println("\nData successfully saved to " + filename + " before and after sorting.");
        // 5. Perform Search operation based on Farmer's name
        System.out.print("\nEnter the name of the farmer you want to search: ");
        String searchName = scanner.nextLine();
        searchFarmer(farmers, searchName);
        scanner.close();
    }
    // Helper method to write data to a text file in a tabular manner
    private static void saveDataToFile(List<Farmer> farmers, String filename, String title, boolean
append) {
        try (PrintWriter writer = new PrintWriter(new FileWriter(filename, append))) {
            writer.println("==========================================================");
            writer.println(title);
            writer.println("==========================================================");
            // Table Header
```

```
        writer.println(String.format("%-15s %-15s %-15s %-10s", "NAME", "FRUIT", "CITY",
"QUANTITY"));

        writer.println("-------------------------------------------------------");

        // Table Data

        for (Farmer f : farmers) {

            writer.println(f.toTableRow());

        }

        writer.println("\n");

    } catch (IOException e) {

        System.out.println("An error occurred while writing to the file: " + e.getMessage());

    }

}

// Helper method to search and display a farmer by name

private static void searchFarmer(List<Farmer> farmers, String searchName) {

    boolean found = false;

    System.out.println("\n--- Search Results ---");

    for (Farmer f : farmers) {

        // Using equalsIgnoreCase so the search is not case-sensitive

        if (f.name.equalsIgnoreCase(searchName)) {

            System.out.println("Farmer Found!");

            System.out.println("Name: " + f.name);

            System.out.println("Fruit: " + f.fruit);

            System.out.println("City: " + f.city);

            System.out.println("Quantity: " + f.quantity);

            found = true;

            break; // Stop searching once found
```

```
            }
        }
        if (!found) {
            System.out.println("No farmer found with the name: " + searchName);
        }
    }
}
```

OUTPUT:-

--- Enter Data for 10 Farmers ---

Farmer 1:

Name: Amit

Fruit sold: Mango

City: Pune

Quantity: 150

Farmer 2:

Name: Rahul

Fruit sold: Apple

City: Shimla

Quantity: 500

Farmer 3:

Name: Sita

Fruit sold: Banana

City: Jalgaon

Quantity: 300

Farmer 4:

Name: Vikram

Fruit sold: Papaya

City: Nashik

Quantity: 100

Farmer 5:

Name: John

Fruit sold: Orange

City: Nagpur

Quantity: 450

Farmer 6:

Name: Maria

Fruit sold: Grapes

City: Sangli

Quantity: 200

Farmer 7:

Name: Ali

Fruit sold: Guava

City: Bhopal

Quantity: 600

Farmer 8:

Name: Priya

Fruit sold: Litchi

City: Muzaffarpur

Quantity: 250

Farmer 9:

Name: Ram

Fruit sold: Pineapple

City: Kochi

Quantity: 120

Farmer 10:

Name: Ken

Fruit sold: Cherry

City: Srinagar

Quantity: 800

Data successfully saved to farmers_data.txt before and after sorting.

Enter the name of the farmer you want to search: Maria

--- Search Results ---

Farmer Found!

Name: Maria

Fruit: Grapes

City: Sangli

Quantity: 200

2. Write a program to take quantity & quality of fruits Mango, Apple & Papaya as input. Quality may be good, bad or average. If entered quantity of fruit is greater than 1000, store it as half. Perform search for a given fruit with given quality.

CODE:-

```
import java.util.ArrayList;

import java.util.List;

import java.util.Scanner;

// Class to represent a Fruit

class Fruit {
```

```
    String name;

    int quantity;

    String quality;

    public Fruit(String name, int quantity, String quality) {

        this.name = name;

        // If entered quantity is greater than 1000, store it as half

        if (quantity > 1000) {

            this.quantity = quantity / 2;

            System.out.println("Note: Quantity for " + name + " was greater than 1000, so it has been
stored as half (" + this.quantity + ").");

        } else {

            this.quantity = quantity;

        }

        this.quality = quality;

    }

}

public class Main {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        List<Fruit> fruits = new ArrayList<>();

        // The three required fruits

        String[] fruitNames = {"Mango", "Apple", "Papaya"};

        System.out.println("--- Enter Fruit Details ---");

        System.out.println("(Quality must be: good, bad, or average)\n");

        // 1. Take input for Mango, Apple, and Papaya

        for (String fruitName : fruitNames) {
```

```
        System.out.println("Entering details for " + fruitName + ":");
        System.out.print("Enter quantity: ");
        int quantity = scanner.nextInt();
        scanner.nextLine(); // Consume newline
        System.out.print("Enter quality (good/bad/average): ");
        String quality = scanner.nextLine().toLowerCase(); // Convert to lowercase for consistency
        fruits.add(new Fruit(fruitName, quantity, quality));
        System.out.println();
    }
    // 2. Perform search operation
    System.out.println("--- Search Fruit ---");
    System.out.print("Enter fruit name to search (Mango/Apple/Papaya): ");
    String searchName = scanner.nextLine();
    System.out.print("Enter quality to search (good/bad/average): ");
    String searchQuality = scanner.nextLine().toLowerCase();
    searchFruit(fruits, searchName, searchQuality);
    scanner.close();
}
// Helper method to search for a fruit by name and quality
private static void searchFruit(List<Fruit> fruits, String name, String quality) {
    boolean found = false;
    System.out.println("\n--- Search Results ---");
    for (Fruit f : fruits) {
        // Check if both name and quality match (ignoring case)
        if (f.name.equalsIgnoreCase(name) && f.quality.equals(quality)) {
```

```
            System.out.println("Match Found!");
            System.out.println("Fruit: " + f.name);
            System.out.println("Stored Quantity: " + f.quantity);
            System.out.println("Quality: " + f.quality);
            found = true;
        }
    }
    if (!found) {
        System.out.println("No matching fruit found for Name: " + name + " with Quality: " + quality);
    }
  }
}
```

OUTPUT:-

```
--- Enter Fruit Details ---
(Quality must be: good, bad, or average)
Entering details for Mango:
Enter quantity: 1200
Enter quality (good/bad/average): good
Note: Quantity for Mango was greater than 1000, so it has been stored as half (600).
Entering details for Apple:
Enter quantity: 500
Enter quality (good/bad/average): average
Entering details for Papaya:
Enter quantity: 1500
Enter quality (good/bad/average): bad
```

Note: Quantity for Papaya was greater than 1000, so it has been stored as half (750).

--- Search Fruit ---

Enter fruit name to search (Mango/Apple/Papaya): Mango

Enter quality to search (good/bad/average): good

--- Search Results ---

Match Found!

Fruit: Mango

Stored Quantity: 600

Quality: good

3. There are four orchards, namely A, B, C & D with fruits in them as Apple, Litchi, Mango, Apple respectively. The qualities of the fruits in the orchards are good, good, good & bad respectively.Search for a given fruit in the available orchards after taking input from the user & print the tree

CODE:-

```
import java.util.ArrayList;

import java.util.List;

import java.util.Scanner;

// Class representing a node in the tree

class OrchardNode {

   String name;

   String fruit;

   String quality;

   List<OrchardNode> children;

   // Constructor for the Root node (doesn't hold fruit itself)

   public OrchardNode(String name) {

      this.name = name;

      this.fruit = "N/A";
```

```
        this.quality = "N/A";
        this.children = new ArrayList<>();
    }
    // Constructor for Orchard nodes (holds fruit and quality data)
    public OrchardNode(String name, String fruit, String quality) {
        this.name = name;
        this.fruit = fruit;
        this.quality = quality;
        this.children = new ArrayList<>();
    }
    // Method to add a child node to this node
    public void addChild(OrchardNode child) {
        this.children.add(child);
    }
}
public class Main {
    public static void main(String[] args) {
        // 1. Initialize the Tree Data
        OrchardNode root = new OrchardNode("All Orchards");
        root.addChild(new OrchardNode("A", "Apple", "good"));
        root.addChild(new OrchardNode("B", "Litchi", "good"));
        root.addChild(new OrchardNode("C", "Mango", "good"));
        root.addChild(new OrchardNode("D", "Apple", "bad"));
        // 2. Print the available orchards in a Tree-View
        System.out.println("--- Available Orchards (Tree-View) ---");
```

```
    printTree(root, "");

    // 3. Take input from the user for searching

    Scanner scanner = new Scanner(System.in);

    System.out.print("\nEnter the fruit you want to search for: ");

    String searchFruit = scanner.nextLine();

    // 4. Create a new root for the searched results

    OrchardNode resultRoot = new OrchardNode("Search Results for '" + searchFruit + "'");

    // Search through the children of the main root

    for (OrchardNode child : root.children) {

        if (child.fruit.equalsIgnoreCase(searchFruit)) {

            resultRoot.addChild(child); // Add matching orchard to the new tree

        }

    }

    // 5. Print the resulting tree

    System.out.println("\n--- Searched Orchards Tree ---");

    if (resultRoot.children.isEmpty()) {

        System.out.println("No orchards found containing: " + searchFruit);

    } else {

        printTree(resultRoot, "");

    }

    scanner.close();

}

// Recursive helper method to print the tree hierarchically

public static void printTree(OrchardNode node, String indent) {

    // Print the current node
```

```
        if (node.fruit.equals("N/A")) {
            System.out.println(indent + "[" + node.name + "]");
        } else {
            System.out.println(indent + "|-- Orchard " + node.name + " (Fruit: " + node.fruit + ", Quality: " +
node.quality + ")");
        }
        // Recursively print all children with increased indentation
        for (OrchardNode child : node.children) {
            printTree(child, indent + "    ");
        }
    }
}
```

OUTPUT:-

```
--- Available Orchards (Tree-View) ---
[All Orchards]
    |-- Orchard A (Fruit: Apple, Quality: good)
    |-- Orchard B (Fruit: Litchi, Quality: good)
    |-- Orchard C (Fruit: Mango, Quality: good)
    |-- Orchard D (Fruit: Apple, Quality: bad)
Enter the fruit you want to search for: Apple
--- Searched Orchards Tree ---
[Search Results for 'Apple']
    |-- Orchard A (Fruit: Apple, Quality: good)
    |-- Orchard D (Fruit: Apple, Quality: bad)
```

4. Take a map provided as a graph & display all the subtrees of connected points.

CODE:-

```
import java.util.ArrayList;

import java.util.HashMap;

import java.util.HashSet;

import java.util.List;

import java.util.Map;

import java.util.Set;

public class Main {

   public static void main(String[] args) {

      // 1. Create a map to represent the graph using an Adjacency List

      Map<String, List<String>> mapGraph = new HashMap<>();

      System.out.println("--- Building the Map Graph ---");

      // Let's create a map with 3 disconnected subtrees (regions)

      // Subtree 1: West/Central India

      addEdge(mapGraph, "Mumbai", "Pune");

      addEdge(mapGraph, "Mumbai", "Surat");

      addEdge(mapGraph, "Pune", "Nagpur");

      // Subtree 2: North India

      addEdge(mapGraph, "Delhi", "Chandigarh");

      addEdge(mapGraph, "Delhi", "Jaipur");

      // Subtree 3: South India (Isolated in this map)

      addEdge(mapGraph, "Chennai", "Bengaluru");

      // Single isolated point (Subtree 4)

      mapGraph.putIfAbsent("Kolkata", new ArrayList<>());

      // 2. Find and display all subtrees (connected components)

      System.out.println("Finding subtrees of connected points...\n");
```

```
    displayAllSubtrees(mapGraph);
}
// Helper method to add a two-way connection (undirected edge)
private static void addEdge(Map<String, List<String>> graph, String point1, String point2) {
    // Ensure both points exist in the map
    graph.putIfAbsent(point1, new ArrayList<>());
    graph.putIfAbsent(point2, new ArrayList<>());
    // Connect them to each other
    graph.get(point1).add(point2);
    graph.get(point2).add(point1);
}
// Method to find and display all connected components
private static void displayAllSubtrees(Map<String, List<String>> graph) {
    Set<String> visited = new HashSet<>();
    int subtreeCount = 1;
    // Loop through every point in the graph
    for (String point : graph.keySet()) {
        // If the point hasn't been visited, it belongs to a new subtree
        if (!visited.contains(point)) {
            List<String> currentSubtree = new ArrayList<>();
            // Use DFS to find all points connected to this one
            dfs(graph, point, visited, currentSubtree);
            // Print the discovered subtree
            System.out.println("Subtree " + subtreeCount + ": " + currentSubtree);
            subtreeCount++;
```

```
        }
    }
  }
  // Depth First Search (DFS) to explore all connected points
  private static void dfs(Map<String, List<String>> graph, String current, Set<String> visited,
List<String> currentSubtree) {
    // Mark current point as visited and add it to the subtree list
    visited.add(current);
    currentSubtree.add(current);
    // Visit all neighboring points
    for (String neighbor : graph.get(current)) {
      if (!visited.contains(neighbor)) {
        dfs(graph, neighbor, visited, currentSubtree); // Recursive call
      }
    }
  }
}
```

OUTPUT:-

```
--- Building the Map Graph ---
Finding subtrees of connected points...
Subtree 1: [Surat, Mumbai, Pune, Nagpur]
Subtree 2: [Chandigarh, Delhi, Jaipur]
Subtree 3: [Kolkata]
Subtree 4: [Chennai, Bengaluru]
```

5. Enter entry point & goal point & produce the matching subtree of the graph.

CODE:-

```
import java.util.ArrayList;

import java.util.Collections;

import java.util.HashMap;

import java.util.HashSet;

import java.util.LinkedList;

import java.util.List;

import java.util.Map;

import java.util.Queue;

import java.util.Scanner;

import java.util.Set;

public class Main {

   public static void main(String[] args) {

      // 1. Create a map to represent the graph

      Map<String, List<String>> mapGraph = new HashMap<>();

      System.out.println("--- Building the Map Graph ---");

      // Creating a network of connected points

      addEdge(mapGraph, "Start", "A");

      addEdge(mapGraph, "Start", "B");

      addEdge(mapGraph, "A", "C");

      addEdge(mapGraph, "B", "D");

      addEdge(mapGraph, "C", "Goal");

      addEdge(mapGraph, "D", "Goal");

      addEdge(mapGraph, "C", "E"); // Dead end

      Scanner scanner = new Scanner(System.in);

      // 2. Take entry point and goal point from the user
```

```
        System.out.print("Enter entry point (e.g., Start): ");
        String entryPoint = scanner.nextLine();
        System.out.print("Enter goal point (e.g., Goal): ");
        String goalPoint = scanner.nextLine();
        // 3. Find and display the matching subtree (path)
        List<String> matchingSubtree = findSubtreePath(mapGraph, entryPoint, goalPoint);
        System.out.println("\n--- Result ---");
        if (matchingSubtree.isEmpty()) {
            System.out.println("No connection found between " + entryPoint + " and " + goalPoint + ".");
        } else {
            System.out.println("Matching Subtree branch found!");
            // String.join connects the list elements with arrows
            System.out.println("Path: " + String.join(" -> ", matchingSubtree));
        }
        scanner.close();
    }
    // Helper method to add a two-way connection (undirected edge)
    private static void addEdge(Map<String, List<String>> graph, String point1, String point2) {
        graph.putIfAbsent(point1, new ArrayList<>());
        graph.putIfAbsent(point2, new ArrayList<>());
        graph.get(point1).add(point2);
        graph.get(point2).add(point1);
    }
    // Uses Breadth-First Search (BFS) to find the path from entry to goal
    private static List<String> findSubtreePath(Map<String, List<String>> graph, String start, String goal)
{
```

```
// If either point doesn't exist in our map, return an empty list
if (!graph.containsKey(start) || !graph.containsKey(goal)) {
    return new ArrayList<>();
}
Queue<String> queue = new LinkedList<>();
Set<String> visited = new HashSet<>();
// This map keeps track of "how we got here" (Child Node -> Parent Node)
// This essentially builds the subtree we use to trace our steps back
Map<String, String> parentMap = new HashMap<>();
queue.add(start);
visited.add(start);
boolean goalFound = false;
// BFS Loop
while (!queue.isEmpty()) {
    String current = queue.poll();
    // Stop searching if we reach the goal
    if (current.equalsIgnoreCase(goal)) {
        goalFound = true;
        // Set the current node to exact casing of the map for reconstruction
        goal = current;
        break;
    }
    // Check all neighboring connected points
    for (String neighbor : graph.get(current)) {
        if (!visited.contains(neighbor)) {
```

```
                visited.add(neighbor);
                parentMap.put(neighbor, current); // Record that we reached 'neighbor' from 'current'
                queue.add(neighbor);
            }
        }
    }
    // Reconstruct the path by tracing parents backwards from the goal
    List<String> path = new ArrayList<>();
    if (goalFound) {
        String step = goal;
        while (step != null) {
            path.add(step);
            step = parentMap.get(step); // Move backwards to the parent
        }
        // Reverse the list so it reads from Start -> Goal
        Collections.reverse(path);
    }
    return path;
  }
}
```

OUTPUT:-

```
--- Building the Map Graph ---
Enter entry point (e.g., Start): Start
Enter goal point (e.g., Goal): Goal
--- Result ---
```

Matching Subtree branch found!

Path: Start -> A -> C -> Goal

6. Give the weightage for each edge of nodes, and produce the shortest path from start to end node.

CODE:-

```
import java.util.ArrayList;

import java.util.Collections;

import java.util.HashMap;

import java.util.List;

import java.util.Map;

import java.util.PriorityQueue;

import java.util.Scanner;

// Class to represent a connection (edge) to another node

class Edge {

   String target;

   int weight;

   public Edge(String target, int weight) {

      this.target = target;

      this.weight = weight;

   }

}

// Class to keep track of current distances in the PriorityQueue

class NodeRecord implements Comparable<NodeRecord> {

   String nodeName;

   int distance;

   public NodeRecord(String nodeName, int distance) {
```

```
        this.nodeName = nodeName;
        this.distance = distance;
    }
    // This ensures the PriorityQueue always serves the node with the shortest distance first
    @Override
    public int compareTo(NodeRecord other) {
        return Integer.compare(this.distance, other.distance);
    }
}
public class Main {
    public static void main(String[] args) {
        // 1. Create the Graph
        Map<String, List<Edge>> graph = new HashMap<>();
        System.out.println("--- Building the Weighted Graph ---");
        // Creating edges with weights (e.g., distances between cities)
        addEdge(graph, "A", "B", 4);
        addEdge(graph, "A", "C", 2);
        addEdge(graph, "B", "C", 5);
        addEdge(graph, "B", "D", 10);
        addEdge(graph, "C", "E", 3);
        addEdge(graph, "E", "D", 4);
        addEdge(graph, "D", "Z", 11);
        // 2. Take input from the user
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter Start Node (e.g., A): ");
```

```
        String startNode = scanner.nextLine().toUpperCase();

        System.out.print("Enter End Node (e.g., Z): ");

        String endNode = scanner.nextLine().toUpperCase();

        // 3. Find and display the shortest path

        findShortestPath(graph, startNode, endNode);

        scanner.close();

    }

    // Helper method to add a two-way weighted connection

    private static void addEdge(Map<String, List<Edge>> graph, String point1, String point2, int weight)
{

        graph.putIfAbsent(point1, new ArrayList<>());

        graph.putIfAbsent(point2, new ArrayList<>());

        graph.get(point1).add(new Edge(point2, weight));

        graph.get(point2).add(new Edge(point1, weight)); // Remove this line if the graph is one-way
(directed)

    }

    // Dijkstra's Algorithm to find the shortest path

    private static void findShortestPath(Map<String, List<Edge>> graph, String start, String end) {

        if (!graph.containsKey(start) || !graph.containsKey(end)) {

            System.out.println("Error: Start or End node does not exist in the graph.");

            return;

        }

        // Keep track of the shortest distance to each node found so far

        Map<String, Integer> distances = new HashMap<>();

        // Keep track of the path (Child -> Parent)

        Map<String, String> previousNodes = new HashMap<>();
```

```
// Priority queue to always explore the closest node next
PriorityQueue<NodeRecord> pq = new PriorityQueue<>();
// Initialize all distances to "infinity", except the start node
for (String node : graph.keySet()) {
    distances.put(node, Integer.MAX_VALUE);
}
distances.put(start, 0);
pq.add(new NodeRecord(start, 0));
while (!pq.isEmpty()) {
    // Get the node with the shortest known distance
    NodeRecord current = pq.poll();
    String currentNode = current.nodeName;
    // If we reached our destination, we can stop searching
    if (currentNode.equals(end)) {
        break;
    }
    // If we found a longer path to a node we've already processed, ignore it
    if (current.distance > distances.get(currentNode)) {
        continue;
    }
    // Check all neighboring connected nodes
    for (Edge edge : graph.get(currentNode)) {
        String neighbor = edge.target;
        int newDist = distances.get(currentNode) + edge.weight;
        // If we found a shorter path to the neighbor, update it
```

```
            if (newDist < distances.get(neighbor)) {
                distances.put(neighbor, newDist);
                previousNodes.put(neighbor, currentNode); // Record how we got here
                pq.add(new NodeRecord(neighbor, newDist)); // Add to queue for future exploration
            }
        }
    }
    // 4. Reconstruct and Print the Path
    printResult(distances, previousNodes, start, end);
  }
  // Helper method to trace back the path and print the results cleanly
  private static void printResult(Map<String, Integer> distances, Map<String, String> previousNodes,
String start, String end) {
    System.out.println("\n--- Shortest Path Result ---");
    if (distances.get(end) == Integer.MAX_VALUE) {
        System.out.println("There is no path connecting " + start + " and " + end + ".");
        return;
    }
    // Trace the path backwards from End to Start
    List<String> path = new ArrayList<>();
    String current = end;
    while (current != null) {
        path.add(current);
        current = previousNodes.get(current);
    }
    // Reverse it so it reads Start -> End
```

```
        Collections.reverse(path);

        System.out.println("Total Cost/Distance: " + distances.get(end));

        System.out.println("Path: " + String.join(" -> ", path));

    }

}
```

OUTPUT:-

```
--- Building the Weighted Graph ---

Enter Start Node (e.g., A): A

Enter End Node (e.g., Z): D

--- Shortest Path Result ---

Total Cost/Distance: 9

Path: A -> C -> E -> D
```

7. Write a program to demonstrate the use of all keywords made for exception handling.

```
import java.util.Scanner;

public class Main {

    // 1. 'throws' keyword: Declares that this method might throw exceptions.

    // It warns the caller that they need to handle these potential errors.

    public static void checkAge(int age) throws SecurityException, ArithmeticException {

        if (age < 18) {

            // 2. 'throw' keyword: Explicitly triggers a new exception object.

            throw new SecurityException("Access Denied: You must be at least 18 years old.");

        } else {

            System.out.println("Access Granted: Welcome!");

        }

    }
```

```
public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.println("--- Exception Handling Demonstration ---");
    System.out.print("Enter your age: ");
    // 3. 'try' keyword: Defines a block of code to monitor for exceptions.
    try {
        int age = scanner.nextInt();
        // This method might throw a SecurityException based on the age
        checkAge(age);
        // This might throw an ArithmeticException if age is 0
        System.out.println("Performing a test calculation (100 / age)...");
        int calculation = 100 / age;
        System.out.println("Result: " + calculation);
    }
    // 4. 'catch' keyword: Captures and handles specific exceptions.
    catch (SecurityException e) {
        System.out.println("\n[Error Caught] " + e.getMessage());
    }
    catch (ArithmeticException e) {
        System.out.println("\n[Error Caught] Math error! Cannot divide by zero.");
    }
    catch (Exception e) {
        // This acts as a fallback for any other unexpected errors (like typing text instead of a number)
        System.out.println("\n[Error Caught] Invalid input detected.");
    }
```

```
        // 5. 'finally' keyword: Code here ALWAYS runs, no matter what happens above.
        finally {
            System.out.println("\n[Finally Block] This always executes. Cleaning up resources...");
            scanner.close();
        }
        System.out.println("Program execution completed successfully.");
    }
}
```

OUTPUT:-Scenario 2: No exceptions occur (The finally block still runs)

```
--- Exception Handling Demonstration ---
Enter your age: 25
Access Granted: Welcome!
Performing a test calculation (100 / age)...
Result: 4
[Finally Block] This always executes. Cleaning up resources...
Program execution completed successfully.
```

OUTPUT:-Exception is thrown (throw) and caught (catch)

```
--- Exception Handling Demonstration ---
Enter your age: 15
[Error Caught] Access Denied: You must be at least 18 years old.
[Finally Block] This always executes. Cleaning up resources...
Program execution completed successfully.
```

8. Write a program to withraw money from a bank account, if the withdrawn money is greater than the balance, then it throws NoMoneyException.

CODE:-

```
import java.util.Scanner;
```

```
// 1. Create a Custom Exception
// We extend the built-in 'Exception' class to make our own specific error
class NoMoneyException extends Exception {
    public NoMoneyException(String message) {
        super(message); // Pass the custom error message to the parent class
    }
}
// 2. Create the Bank Account class
class BankAccount {
    private double balance;
    // Constructor to set initial balance
    public BankAccount(double initialBalance) {
        this.balance = initialBalance;
    }
    // Method to withdraw money. Notice the 'throws' keyword warning about our custom exception
    public void withdraw(double amount) throws NoMoneyException {
        System.out.println("Attempting to withdraw: Rs. " + amount);
        // Check if withdrawal amount is greater than the balance
        if (amount > balance) {
            // Trigger our custom exception with a specific error message
            throw new NoMoneyException("Transaction Failed: Not enough money. Current balance is Rs.
" + balance);
        }
        // If no exception was thrown, proceed with the deduction
        balance -= amount;
        System.out.println("Transaction Successful! Remaining balance: Rs. " + balance);
```

```
    }
}
public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        // Initialize a bank account with Rs. 5000
        BankAccount myAccount = new BankAccount(5000.00);
        System.out.println("--- Welcome to the Bank ---");
        System.out.println("Current Balance: Rs. 5000.00\n");
        System.out.print("Enter amount to withdraw: Rs. ");
        double withdrawAmount = scanner.nextDouble();
        // 3. Try the withdrawal and catch the custom exception if it happens
        try {
            myAccount.withdraw(withdrawAmount);
        } catch (NoMoneyException e) {
            // This block runs if the NoMoneyException is thrown
            System.out.println("\n[ERROR] " + e.getMessage());
        } finally {
            System.out.println("\nThank you for banking with us!");
            scanner.close();
        }
    }
}
```

Output:-Scenario 1: Successful Withdrawal

```
--- Welcome to the Bank ---
```

Current Balance: Rs. 5000.00

Enter amount to withdraw: Rs. 2000

Attempting to withdraw: Rs. 2000.0

Transaction Successful! Remaining balance: Rs. 3000.0

Thank you for banking with us!

Output:-SCENERIO 2 Triggering the Custom Exception

--- Welcome to the Bank ---

Current Balance: Rs. 5000.00

Enter amount to withdraw: Rs. 7500

Attempting to withdraw: Rs. 7500.0

[ERROR] Transaction Failed: Not enough money. Current balance is Rs. 5000.0

Thank you for banking with us!

9. Write a program which enables to use a platform by a train one at a time.

Output:-

```
// 1. The Shared Resource
class Platform {
   // The 'synchronized' keyword ensures ONLY ONE thread (train) can run this method at a time
   public synchronized void usePlatform(String trainName) {
      System.out.println(">> " + trainName + " has arrived at the platform.");
      try {
         System.out.println(trainName + " is boarding passengers...");
         // Simulate the train stopped at the platform for 2 seconds
         Thread.sleep(2000);
      } catch (InterruptedException e) {
         System.out.println("Train interrupted.");
```

```
        }
        System.out.println("<< " + trainName + " has left the platform.\n");
    }
}
// 2. The Thread Class
class Train extends Thread {
    Platform platform;
    String trainName;
    // Constructor assigns the shared platform and the train's name
    public Train(Platform platform, String trainName) {
        this.platform = platform;
        this.trainName = trainName;
    }
    // This method runs when we start the thread
    @Override
    public void run() {
        // The train attempts to use the platform
        platform.usePlatform(trainName);
    }
}
// 3. Main Execution
public class Main {
    public static void main(String[] args) {
        // We only have ONE shared platform
        Platform sharedPlatform = new Platform();
```

```
        // Create multiple trains that all want to use the SAME platform
        Train train1 = new Train(sharedPlatform, "Express Train 101");
        Train train2 = new Train(sharedPlatform, "Local Train 202");
        Train train3 = new Train(sharedPlatform, "Shatabdi Express 303");
        System.out.println("--- Train Station Operations Started ---\n");
        // Start all train threads at roughly the same time
        train1.start();
        train2.start();
        train3.start();
    }
}
```

Output:-

```
--- Train Station Operations Started ---
>> Express Train 101 has arrived at the platform.
Express Train 101 is boarding passengers...
<< Express Train 101 has left the platform.
>> Shatabdi Express 303 has arrived at the platform.
Shatabdi Express 303 is boarding passengers...
<< Shatabdi Express 303 has left the platform.
>> Local Train 202 has arrived at the platform.
Local Train 202 is boarding passengers...
<< Local Train 202 has left the platform.
```

10. Write a programme of Restaurant and Waiter, which accept order if kitchen

Code:-

```
// 1. The Shared Resource (The Restaurant/Kitchen)
```

```
class Restaurant {
    // If true, the kitchen is busy with an order. If false, the kitchen is free.
    private boolean isKitchenBusy = false;
    // Waiter calls this method to place an order
    public synchronized void acceptOrder(int orderNumber) {
        // If the kitchen is already busy, the Waiter must WAIT
        while (isKitchenBusy) {
            try {
                wait();
            } catch (InterruptedException e) {
                System.out.println("Waiter was interrupted.");
            }
        }
        // Kitchen is free, so Waiter places the order
        System.out.println("Waiter: Accepted Order #" + orderNumber + " and sent to Kitchen.");
        isKitchenBusy = true; // Kitchen is now busy
        // NOTIFY the Chef that an order is ready to be prepared
        notify();
    }
    // Chef calls this method to cook and deliver
    public synchronized void deliverOrder() {
        // If there are no orders, the Chef must WAIT
        while (!isKitchenBusy) {
            try {
                wait();
```

```
        } catch (InterruptedException e) {
            System.out.println("Chef was interrupted.");
        }
    }
    // Chef prepares and delivers the order
    System.out.println("Chef  : Order cooked and delivered! Kitchen is now free.\n");
    isKitchenBusy = false; // Kitchen is now free again
    // NOTIFY the Waiter that the kitchen is ready for a new order
    notify();
  }
}
// 2. Waiter Thread
class Waiter extends Thread {
  Restaurant restaurant;
  public Waiter(Restaurant restaurant) {
    this.restaurant = restaurant;
  }
  @Override
  public void run() {
    // Waiter takes 5 orders
    for (int i = 1; i <= 5; i++) {
        restaurant.acceptOrder(i);
        try {
            Thread.sleep(500); // Waiter takes a moment before taking the next order
        } catch (InterruptedException e) {}
```

```
        }
    }
}
// 3. Chef Thread
class Chef extends Thread {
    Restaurant restaurant;
    public Chef(Restaurant restaurant) {
        this.restaurant = restaurant;
    }
    @Override
    public void run() {
        // Chef delivers 5 orders
        for (int i = 1; i <= 5; i++) {
            restaurant.deliverOrder();
            try {
                Thread.sleep(1000); // It takes the Chef some time to cook
            } catch (InterruptedException e) {}
        }
    }
}
// 4. Main Execution
public class Main {
    public static void main(String[] args) {
        Restaurant sharedRestaurant = new Restaurant();
        Waiter waiter = new Waiter(sharedRestaurant);
```

```
        Chef chef = new Chef(sharedRestaurant);

        System.out.println("--- Restaurant Opened ---\n");

        // Start both threads

        waiter.start();

        chef.start();

    }

}
```

Output:-

```
--- Restaurant Opened ---

Waiter: Accepted Order #1 and sent to Kitchen.

Chef  : Order cooked and delivered! Kitchen is now free.

Waiter: Accepted Order #2 and sent to Kitchen.

Chef  : Order cooked and delivered! Kitchen is now free.

Waiter: Accepted Order #3 and sent to Kitchen.

Chef  : Order cooked and delivered! Kitchen is now free.

Waiter: Accepted Order #4 and sent to Kitchen.

Chef  : Order cooked and delivered! Kitchen is now free.

Waiter: Accepted Order #5 and sent to Kitchen.

Chef  : Order cooked and delivered! Kitchen is now free.
```